REPORT

byn17Z

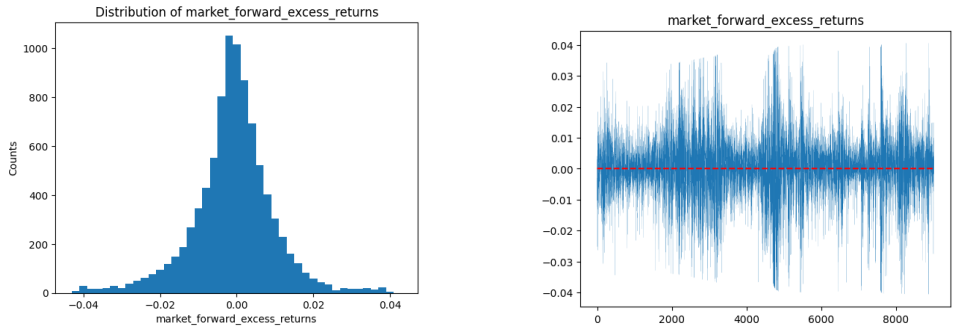
1 INTRODUCTION

Data science has changed the way people explore the stock market. With enough features and machine learning methods, it's possible to uncover the repeatable patterns of the market. The Kaggle competition requires participants to build a model that predicts excess returns of the S&P 500 index using a tailored set of market data features (Tactical et al., 2025). This project aims to explore the underlying patterns in the market and model the trend of excess returns from the S&P 500 index based on Recurrent Neural Network (RNN) and Transformer encoder models, and try to generate an allocation strategy for the S&P 500 index.

2 DATA EXPLORATION & FEATURE ENGINEERING

2.1 DISTRIBUTION OF TARGET VARIABLES

The distribution of the excess returns is similar to a normal distribution with mean 0.



(a) Distribution (b) Trend

Figure 1: Market Forward Excess Returns

2.2 CORRELATION COEFFICIENT

Pearson correlation coefficient (PCC) is a normalized measurement of linear correlation between two sets of data (Wikipedia contributors, 2025). For random vectors $(x^{(1)}, x^{(2)}, ..., x^{(n)})$, the PCC between 2 RVs is defined as

$$r_{\mathbf{x}^{(i)},\mathbf{x}^{(j)}} = \frac{\mathrm{Cov}(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})}{\sqrt{\mathrm{Var}(\mathbf{x}^{(i)})\mathrm{Var}(\mathbf{x}^{(j)})}}, \qquad i, j = 1, 2, ..., n \tag{1}$$

It is the ratio between the covariance of two variables and the product of their standard deviations; thus, the result always has a value between -1 and 1. The closer the absolute value $|r_{\mathbf{x}^{(i)},\mathbf{x}^{(j)}}|$ is to 1, the more linear correlation is implied between the two random variables (Blitzstein & Hwang, 2019).

The correlation coefficient between each input feature and the excess return value is calculated as shown in Figure 2

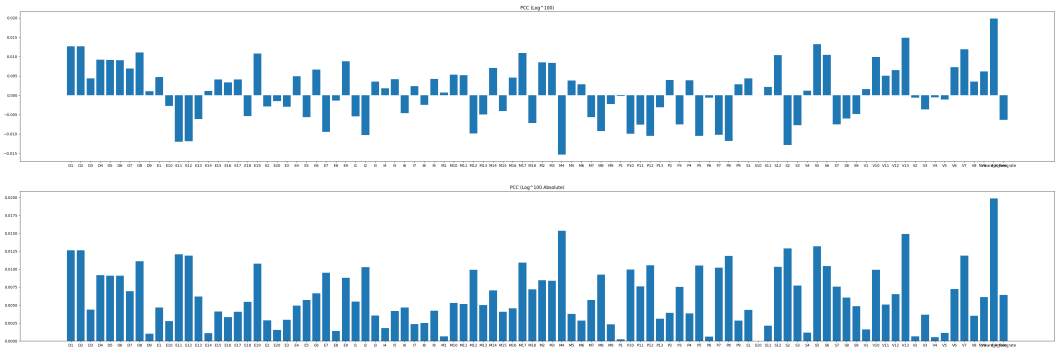


Figure 2: PCC

Table 1: Statistics of $|PCC|$ (Absolute Value)

AVERAGE	MAX	25-PERCENTILE	MEDIAN	75-PERCENTILE	STD
0.0119	0.0647	0.0037	0.0073	0.0171	0.0118

Features with $|r_{\mathbf{x}^{(i)},\mathbf{y}}| \leq 25 percentile$ (lowest 25%, Table 1) are dropped for little significance to the excess return value.

2.3 Missing Values Handling

For each feature column in the dataset, count the number of missing values and find out those with proportion of values missing higher than 20%, as shown in Table 2. These features are dropped for completeness and correctness concerns. Then, interpolate by each features with linear functions.

Table 2: Missing Values ($\geq 20\%$)

FEATURE	COUNT	PROPORTION
E7	6969	77.2531%
M1	5547	61.4899%
M13	5540	61.4123%
M14	5540	61.4123%
M2	3217	35.6612%
M3	2018	22.3700%
M5	3283	36.3929%
M6	5043	55.9029%
S12	3537	39.2085%
S3	5733	63.5517%
S8	3009	33.3555%
V10	6049	67.0547%
V9	4539	50.3159%

3 Model Selection & Training

3.1 Dataset Reconstruction

To deal with time series data, recurrent models seem to be good choices for carrying preceding information to determine a new prediction. However, the sequence of preceding data should not be too long; otherwise, the dimension of the input tensor would be too large, causing a huge computational cost and memory usage. To prevent the explosion of computational cost, the sequence length of

preceding data applied to each day's prediction is set to be a constant SEQ_LEN. Therefore, the size of input tensors are zipped into $(SEQ_LEN \times FEATURES_NUM)$.

3.2 Model Structures

3.2.1 Recurrent Models

Double layer recurrent models, RNN, GRU, and LSTM, are built in one constructor interface with hidden layers dimension 64. By inputting a sequence of historical time series data, the model outputs an updated hidden state and a primary output of dimension 64×1 which will be transformed into a 1-dimensional prediction for the excess return by a feedforward linear network (Hochreiter & Schmidhuber, 1997).

3.2.2 Transformer Encoder Model

The 2-layer Transformer encoder model in this project consists of a position embedding layer, two 4-head Attention layers of dimension 64 with a 128-dimensional feedforward layer for each respectively, and a 64-to-1 MLP that gives the final predictions (Vaswani et al., 2017).

3.3 Model Training and Validation

3.3.1 Training Process

Stochastic Gradient Descent is applied in model training to enhance training speed and reduce memory usage given the large scale of the training dataset. With size 256 for each batch and input tensor size 50×62 for each sample, the GPU memory usage reaches a maximum of about 3 GB.

Randomness occurs during model training due to the random shuffle of the training set, leading to fluctuating training results.

3.3.2 Validation Methods

To evaluate how well a model performs both on training data and unseen data, K-fold cross validation is applied to measure model performance and tune hyper-parameters. A case result of K-fold cross-validation based on some preset parameters is shown in Table 3 and Figure 3.

Table 3: K-Fold Cross-Validation

MODEL	AVERAGE LOSS	AVERAGE VARIANCE
RNN	1.85e-4	3.25e-5
GRU	2.19e-4	3.14e-5
LSTM	2.18e-4	3.26e-5
Transformer	1.24e-4	0.83e-5

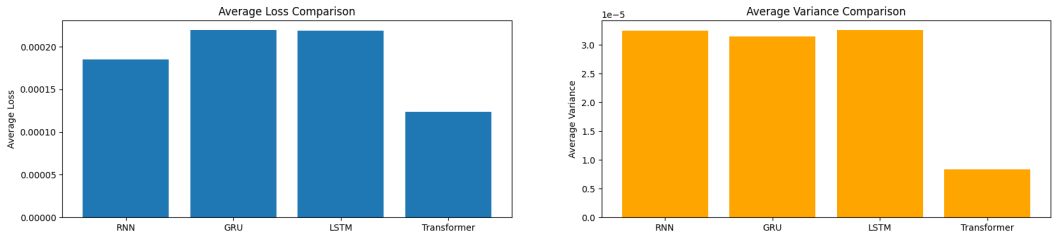


Figure 3: K-Fold Cross-Validation

3.4 HYPER-PARAMETER CHOICES

In order to figure out appropriate hyper-parameters, an optimization algorithm is needed. However, taking derivative with respect to a hyper-parameter is not always an easy work. Therefore, a simplified variation of **Bisection method** with no derivatives involved is applied to find a small enough in a bounded interval.

Consider a smooth function f on $[a_0, b_0]$. In each iteration, $c_k = \frac{a_k+b_k}{2}$, $l_k = \frac{a_k+c_k}{2}$, $r_k = \frac{a_k+c_k}{2}$

$$\text{if } f(l_k) \leq f(r_k), \quad a_{k+1} = a_k, \quad b_{k+1} = c_k;$$

$$\text{if } f(l_k) \geq f(r_k), \quad a_{k+1} = c_k, \quad b_{k+1} = b_k.$$

Then $|a_k - b_k| = (b_0 - a_0)/2^k \to 0$ while $f(c_k)$ decreasingly converges.

3.4.1 LEARNING RATE

Learning rate scaler determine how far a step of gradient descend go, which affects not only convergence speed but also accuracy. In order to figure out an appropriate learning rate value, a simplified bisection method is applied in $[0.0005, 0.05]$ with K-fold cross-validation as measure. However, implementing K-fold cross-validation is a huge amount of computation. So l_k and r_k are substituted by a_k and b_k respectively. (One case of tuning result is shown in Figure 4.)

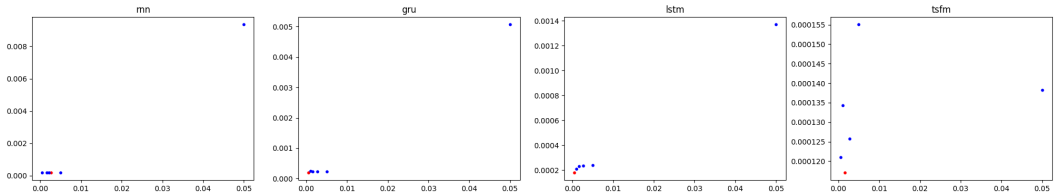


Figure 4: Learning Rate

3.4.2 PRECEDING DATA LENGTH

The length of the preceding data to which the model can refer seems to be important to the model's capability by intuition. It's a trade-off between information gain and computational cost. So a bisection optimization is implemented on preceding sequence length between $[20, 100]$ as done before. (One case of tuning result is shown in Figure 5.)

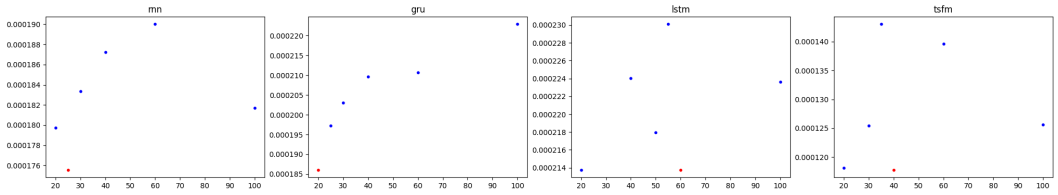


Figure 5: Preceding Length

3.4.3 MODEL CHOICE

Training and validation of 4 model types with K-fold cross-validation on train set. Compare the losses and variances. (Figure 6)

Test the K-fold trained models on test set. Compare the losses and variances. (Figure 7)

The outputs show that:

1. Transformer model has lowest average loss.

2. Transformer model has significantly average variance than other type models;

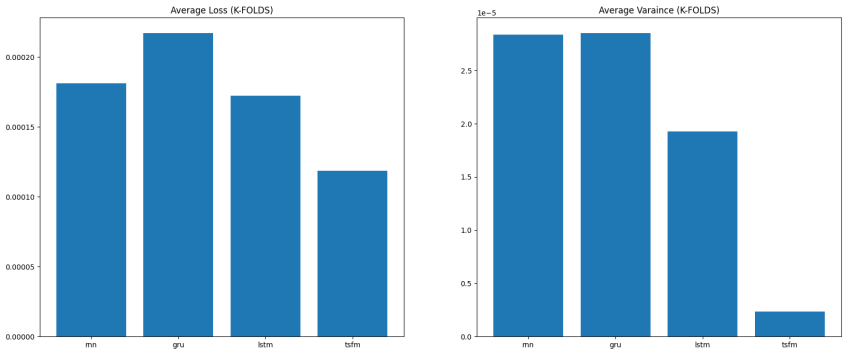


Figure 6: Comparison of 4 Model Types on Train Set (Loss & Variance)

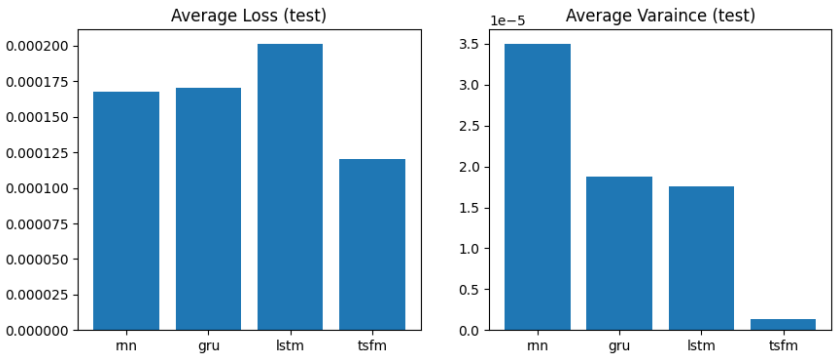


Figure 7: Comparison of 4 Model Types on Test Set (Loss & Variance)

3. Performance of Transformer model is much more stable on K-folds than other type models;

4. The Transformer model seems to be less likely to overfit.

3.4.4 Allocation Function

A clip function is applied with a scaler and a 0.005 bias to transform the predicted excess returns values into an allocation between 0 and 2.

$$\textbf{Allocation} = clip_{[0,2]}(scaler \cdot (prediction + bias)) \tag{2}$$

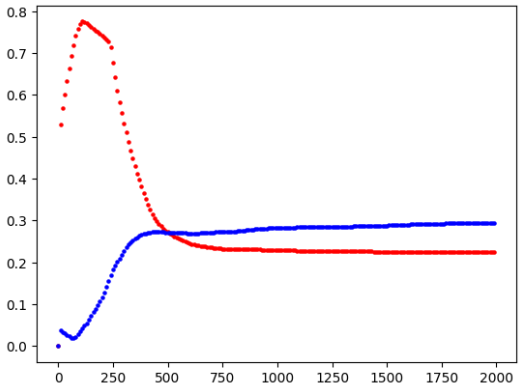
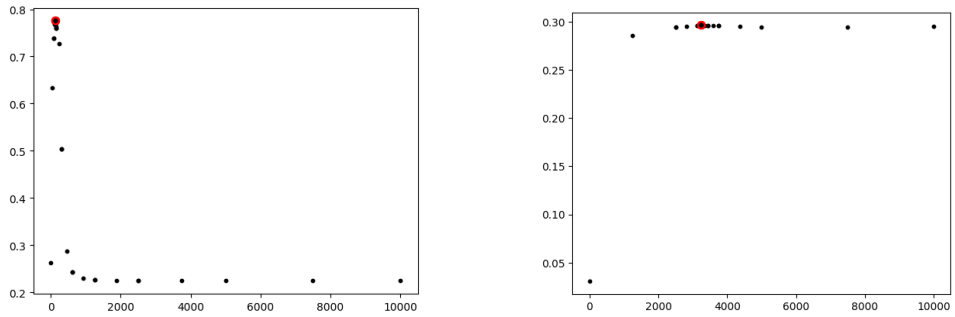


Figure 8: Scaler - Score (Transformer: red; LSTM: blue)



(a) Transformer (b) LSTM

Figure 9: Tuning Process - Scaler

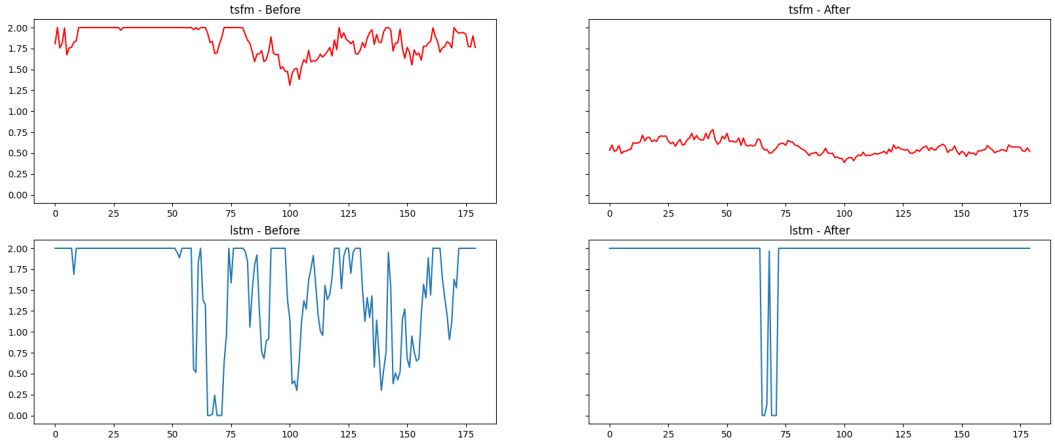


Figure 10: Test Allocation Function (Scaler tuned: before → after)

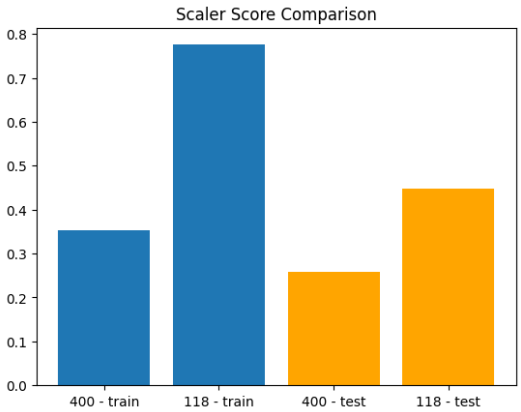


Figure 11: Scaler Score on Train and Test Sets

With the scaler value determined, the score of the Transformer Model on train set has been greatly improved; but on test set the improvement is much smaller (Figure 11). Overfitting still exists.

4 EVALUATION

During the 10 times new model training, LSTM models performance better and more stable on the train set than the Transformer models. But on the test set the case inverses. Transformer models has lower and more stable MSE Losses and higher scores on test set, which implies a better capability to perform on new unseen data.

Transformer models with selected hyper-parameters get an average score 0.425 and standard deviation 0.111 in the 10 model train runs on the local test set (last 180 days of the given dataset). (Figure 12)

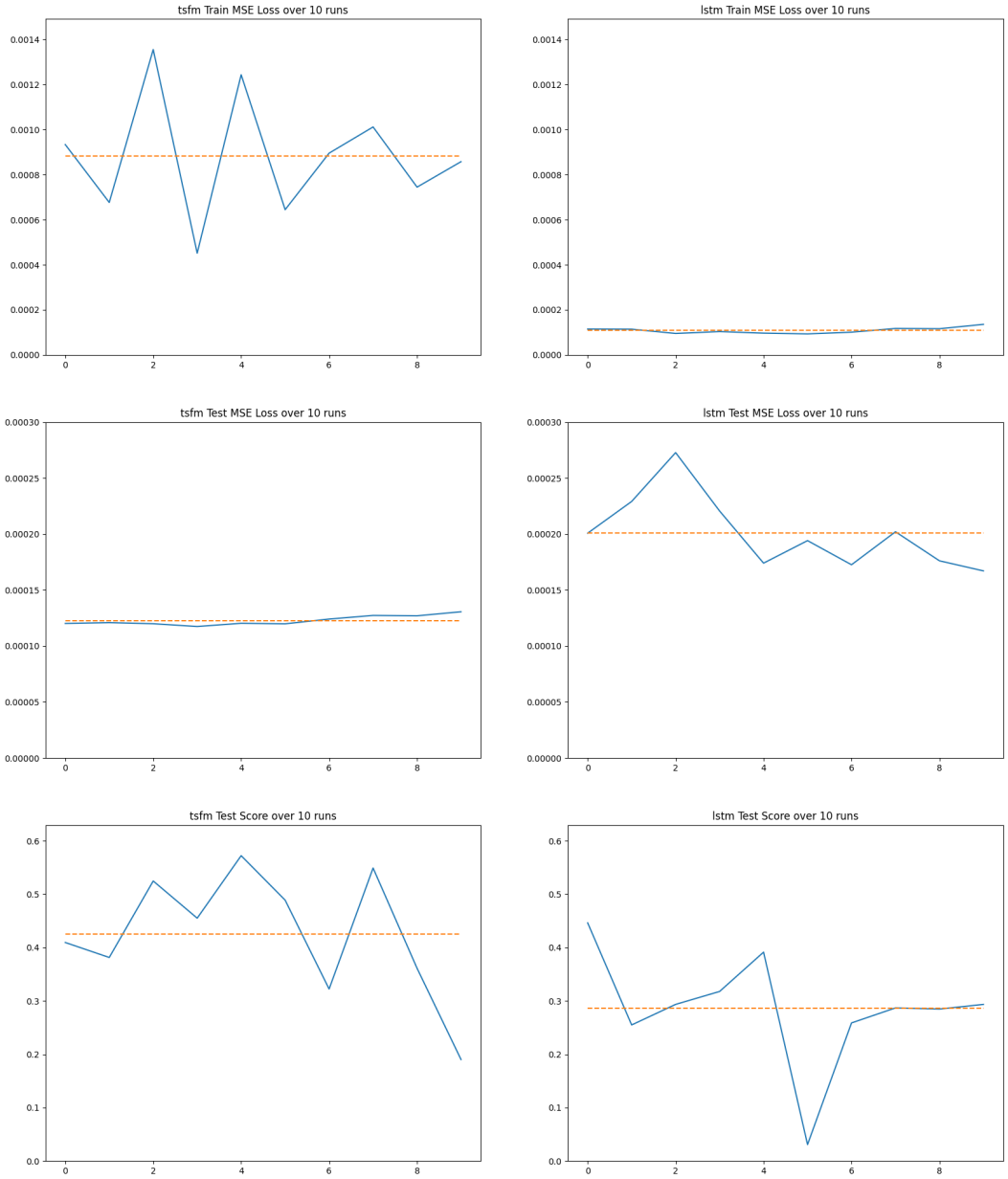


Figure 12: MSE Loss and Scores over 10 times Model Training

By calling Kaggle interface server, the Transformer models with selected hyper-parameters get an average score 0.457 and standard deviation 0.238 from the server, as shown in Figure 13

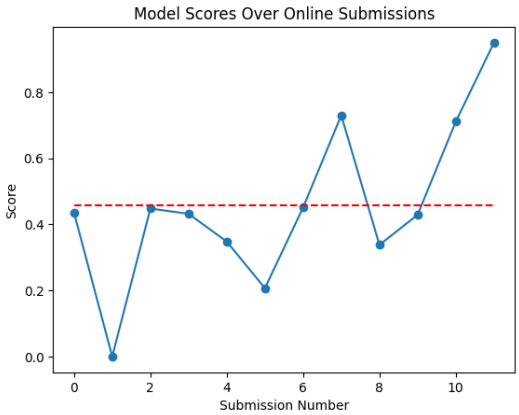


Figure 13: Online Submission Scores

Given the results above, it's clear that the volatility of the final scores is significantly high while the scores are not so good, fluctuating between 0.2 and 0.9.

5 Conclusion

The results have proven the potential of recurrent models and Transformer models to predict the market and perform reasonable strategy with appropriate hyper-parameters choices. However, the RNN models still face overfitting problems, and the scores attained by the Transformer models are too unstable and have much randomness. And the optimizer method for hyper-parameters tuning may still raise convergent issues. To achieve more robust and reliable results, there's still much work to do.

References

Joseph K Blitzstein and Jessica Hwang. *Introduction to probability*, chapter 7.3. Chapman and Hall/CRC, 2019.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8): 1735–1780, 1997.

Hull Tactical, Addison Howard, Laurent Lanteigne, and Petra Bakosova. Hull tactical - market prediction. `https://www.kaggle.com/competitions/hull-tactical-market-prediction/overview`, 2025. [Online; accessed 9-December-2025].

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Wikipedia contributors. Pearson correlation coefficient — Wikipedia, the free encyclopedia. `https://en.wikipedia.org/w/index.php?title=Pearson_correlation_coefficient&oldid=1320577835`, 2025. [Online; accessed 9-December-2025].